MODULE – 2

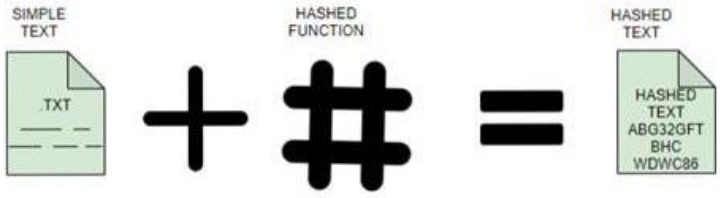
Cryptography for Blockchain Security: Hash functions (SHA-256, Keccak), Public key cryptography (RSA, ECC), Digital signatures & PKI in blockchain, Merkle trees & Patricia tries, Wallets and key management.

Hash functions (SHA-256, Keccak)

SHA-256 (Secure Hash Algorithm 256-bit)

The SHA-256 is a hashing algorithm, this algorithm is very widely known and popularly used in the security as well as the cryptography of modern computers, this algorithm is used to protect sensitive data as well as maintain the security of the systems so in this article.

Hashing is the process of scrambling raw information and scrambling it to such an extent that reproducing it back to the original form that it was in is not possible so in simple words, hashing simply takes a piece of the information and then it passes it through something called a hash function and the hash function performs some mathematical operations on the raw data.



As you can see in the above image, we are using hash function that is converting the plain text into a hash digest these types of hash digests are irreversible which means that no one can provide you the original data by any means.Now that we have some basic understanding of hashing let's understand what is SHA-256 algorithm.

SHA 256 is a part of the SHA 2 family of algorithms, where SHA stands for Secure Hash Algorithm. Published in 2001, it was a joint effort between the NSA and NIST to introduce a successor to the SHA 1 family, which was slowly losing strength against brute force attacks.

The significance of the 256 in the name stands for the final hash digest value, i.e. irrespective of the size of plaintext/cleartext, the hash value will always be 256 bits.

Characteristics of the SHA-256

The SHA algorithm has several important features, like −

- **Message length** − The cleartext should be less than 264 bits. The size has to be in the same range to keep the digest as random as possible.

- **Digest Length** − The hash digest should be 256 bits for the SHA 256 method, 512 bits for SHA-512, and so on. Higher digests typically represent considerably more calculations at a cost of speed and space.

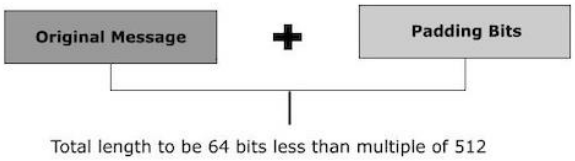
- **Irreversible** − By design, all hash functions, like SHA 256, are irreversible. You should not get a plaintext if you already have the digest, nor should the digest return its original value if you put it over the hash function again.

Algorithm

You can divide the whole procedure into five different sections, as shown below −

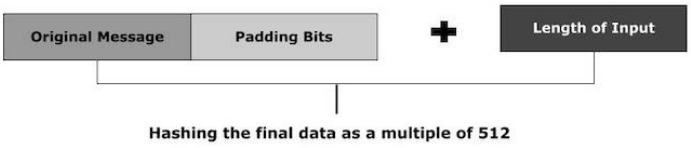
- **Padding Bits**

It gives some extra bits to the message so that the length is precisely 64 bits less than a multiple of 512. In addition, the first bit must be one, and the remaining bits should be zeros.



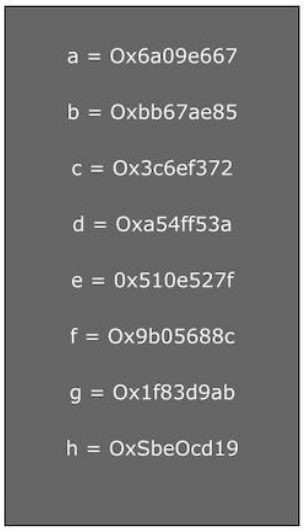
- **Padding Length**

We can now add 64 bits of data to the final plaintext, which makes it a multiple of 512. To determine these 64 bits of characters, apply the modulus to the initial plaintext without padding.



Initialising the Buffers

The default settings for the eight buffers that will be used in the rounds should be set as follows −



We need to keep 64 different keys in an array, ranging from K[0] to K[63].

Compression Functions

The entire message is divided into numerous blocks of 512 bits each. It runs each block over 64 rounds of operations, with the output of each one being used as the input to the next block.

Since the value of K[i] in all of those iterations is pre-initialized, W[i] is another input that is calculated separately for each block according to the number of iterations being performed at the time.

Output

With each iteration, the block's final output becomes the input for the next block. The entire cycle is carried out until we reach the last 512-bit block, at the point where the output is considered the final hash digest. As the algorithm's name suggests, this digest will be 256 bits in length.

Keccak-256

Keccak-256 is a cryptographic hash function that is part of the Keccak family, which was selected as the winner of the NIST (National Institute of Standards and Technology) SHA-3 competition. Keccak-256 produces a fixed 256-bit (32-byte) hash output and is known for its unique design and enhanced security features.

Key Features

- **Sponge Construction:** Unlike traditional hash functions, Keccak employs a sponge construction, which allows it to absorb input data and then squeeze out the hash output. This structure contributes to its flexibility and efficiency.
- **Variable Output Size:** While Keccak-256 produces a 256-bit output, the Keccak family allows for variable-length outputs, making it adaptable for different use cases.

- **High Security:** Keccak-256 is designed to be resistant to various cryptographic attacks, including collision, pre-image, and second pre-image attacks.
- **Performance:** The sponge construction allows for parallel processing, making Keccak-256 efficient on various hardware, including resource-constrained environments.

Use Cases

- **Blockchain Technology:** Keccak-256 is prominently used in Ethereum, where it serves as the basis for hashing transactions, blocks, and smart contract data.
- **Digital Signatures:** It is employed in various digital signature schemes, ensuring the integrity and authenticity of messages.
- **Data Integrity:** Similar to other hash functions, Keccak-256 is used to verify the integrity of data in applications where security is paramount.

How Does Keccak-256 Work?

Here is an overview of the process:

- **Input Preparation:** The input message is padded with a specific delimiter to ensure it fits the sponge's requirements.
- **State Initialization:** The process begins with a 1600-bit state initialized to zero.
- **Absorption Phase:** The padded input is absorbed into the state in blocks, with each block processed and mixed into the state using a permutation function.
- **Permutation Function:** The state undergoes multiple rounds of transformations that mix and diffuse the input data, enhancing security.
- **Squeezing Phase:** After absorbing the input, the output is extracted from the state. Keccak-256 produces a 256-bit hash output.

Cryptography

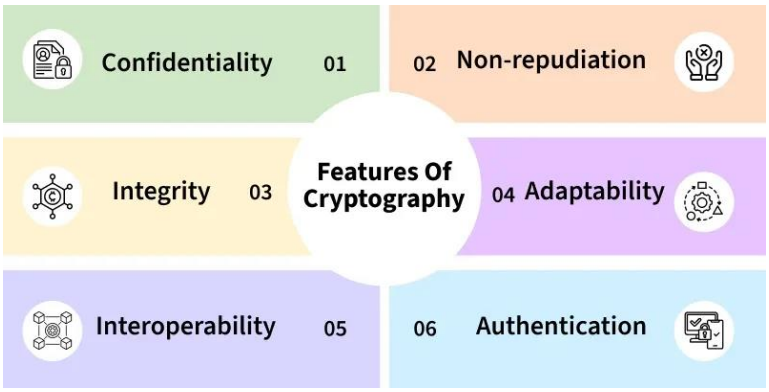
Cryptography is the science of protecting information using mathematical techniques to ensure confidentiality, integrity, and authentication. It transforms readable data into unreadable form, preventing unauthorized access and tampering.

- Converts plaintext into ciphertext using algorithms and keys
- Ensures confidentiality, integrity, authentication, and non-repudiation
- Used in secure communication, digital signatures, passwords, and online transactions

These algorithms are used for cryptographic key generation, digital signing, and verification to protect data privacy, web browsing on the internet and to protect confidential transactions such as credit card and debit card transactions.

Features Of Cryptography

The features of cryptography that makes it a popular choice in various applications could be listed down as



1. **Confidentiality:** Information can only be accessed by the person for whom it is intended and no other person except him can access it.
2. **Non-repudiation:** The creator/sender of information cannot deny his intention to send information at a later stage.
3. **Integrity:** Information cannot be modified in storage or transition between sender and intended receiver without any addition to information being detected.
4. **Adaptability:** Cryptography continuously evolves to stay ahead of security threats and technological advancements.
5. **Interoperability:** Cryptography allows for secure communication between different systems and platforms.
6. **Authentication:** The identities of the sender and receiver are confirmed. As well destination/origin of the information is confirmed.

Types Of Cryptography

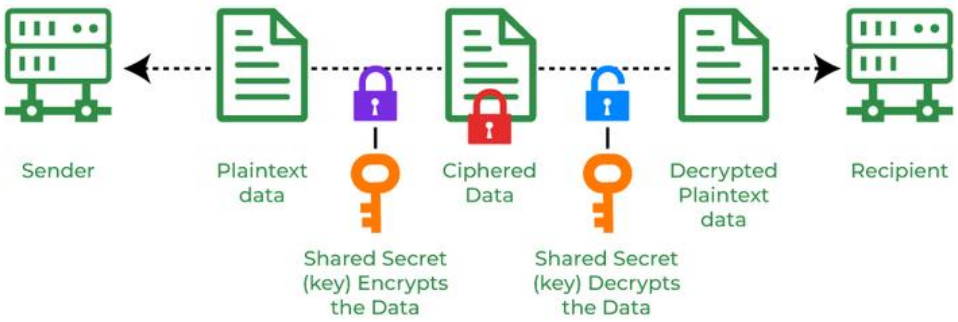
There are three types of cryptography, namely **Symmetric Key Cryptography**, **Asymmetric Key Cryptography** and **Hash functions.**

Symmetric Key Cryptography

Symmetric Key Cryptography is an encryption system where the sender and receiver of a message use a single common key to encrypt and decrypt messages.

Symmetric Key cryptography is faster and simpler but the problem is that the sender and receiver have to somehow exchange keys securely.

The most popular symmetric key cryptography systems are **Data Encryption Systems (DES)** and **Advanced Encryption Systems (AES).**



Asymmetric Key Cryptography (PUBLIC KEY CRYPTOGRAPHY)

In Asymmetric Key Cryptography a pair of keys is used to encrypt and decrypt information. A sender's public key is used for encryption and a receiver's private key is used for decryption. Public keys and Private keys are different. Even if the public key is known by everyone the intended receiver can only decode it because he holds his private key. The most popular asymmetric key cryptography algorithm is the RSA algorithm.

Hash Functions

Hash functions do not require a key. Instead, they use mathematical algorithms to convert messages of any arbitrary length into a fixed-length output, known as a hash value or digest.
Hash functions are designed to be one-way, meaning the original input cannot be derived from the output.

RSA Algorithm

RSA(Rivest-Shamir-Adleman) Algorithm is an asymmetric or public-key cryptography algorithm which means it works on two different keys: Public Key and Private Key. The Public Key is used for encryption and is known to everyone, while the Private Key is used for decryption and must be kept secret by the receiver. RSA Algorithm is named after Ron Rivest, Adi Shamir and Leonard Adleman, who published the algorithm in 1977.

Example of Asymmetric Cryptography:

If Person A wants to send a message securely to Person B:

Person A encrypts the message using Person B's Public Key.
Person B decrypts the message using their Private Key.

*RSA Algorithm is based on **factorization** of large number and **modular arithmetic** for encrypting and decrypting data. It consists of three main stages:*

1. ***Key Generation:** Creating Public and Private Keys*
2. ***Encryption:** Sender encrypts the data using Public Key to get **cipher text**.*
3. ***Decryption:** Decrypting the **cipher text** using Private Key to get the original data.*

Let's look at the RSA Algorithm Steps:

i. You can choose any two large prime numbers, say A and B.
ii. Next, you can find the product of A and B, say N.
    N = A*B
iii. You have to select a public key, say E, for encryption. You have to make sure this key isn't a factor of (A-1) and (B-1).
iv. Now you can select the private key for decryption say, D. The private key should match this equation:
    (D*E) mod (A-1)*(B-1) = 1
v. You can calculate the ciphertext from the plaintext using this equation:
    Ciphertext = Plaintext^E mod N
vi. Once the ciphertext is generated, it should be sent to the recipient.
vii. You can decrypt the plaintext from the ciphertext using this equation:
    Plaintext = Ciphertext^D mod N

RSA Algorithm Example

A = 7, and B = 17

N = A*B

N = 7*17

N = 119

Now, we have to select a public key, say p, so that isn't a factor of (A-1) and (B-1).

(7-1)*(17-1) = 6*16 = 96

Now, the factor of 96 is 2*2*2*2*2*3.

So, we can choose our public key, E as 5 since it isn't a factor of 2 or 3.

Now, to select the private key, we have to make sure it matches this equation:

(D*E) mod (A-1)*(B-1) = 1

(D*5) mod (6)*(16) = 1

(D*5) mod 96 = 1

Now, we can choose D as 77 to satisfy the equation.

(77*5) mod 96 = 1

385 mod 96 = 1

1 = 1

It satisfies the equation so we can move on.

Now, we have to calculate the ciphertext.

Let's take our plaintext to be 10.

Ciphertext = Plaintext ^ E mod N

Ciphertext = 10^5 mod 119

Ciphertext = 40

Once we have the ciphertext, we can send it to the recipient.

We can also check the value by decrypting it with the equation:

Plaintext = Ciphertext ^ D mod N

Plaintext = 40^77 mod 119

Plaintext = 10

That is the plaintext we chose.

Application of RSA Algorithm

Let's look at some applications of the RSA Algorithm:

- Sending Information Safely: RSA is used to protect important data when it's sent over the internet, so only the right person can read it.
- Digital Signatures: It lets people "sign" documents or messages in a digital way, so others can be sure it came from them and wasn't changed.
- Secure Websites (HTTPS): RSA helps websites create a safe connection with your browser, which is why you see a lock icon when visiting secure websites.
- Safe Email Communication: RSA is used in tools like PGP to keep your emails private and also to prove who sent them.
- Logging into Remote Computers (SSH): It helps people securely connect to another computer from far away, like IT admins working on servers.

Possible Attacks on RSA Algorithm

Here's a list of the possible attacks on the RSA Algorithm:

1. Plaintext Attack

A plaintext attack is when a hacker has some original messages (plaintext) and their encrypted versions (ciphertext). The hacker tries to study these pairs to find out the encryption key or figure out how to decode other messages without needing the key.

There can be three types of Plaintext Attacks:

a. Short Message Attack

In short message attacks, it is generally assumed that the attacker already knows some of the plaintext messages. Now, if an attacker knows some blocks of plaintext, they could try to encrypt the blocks using the information. Padding bits of encryption is used to prevent a short message attack.

b. Cycling attack

The reverse process takes place in a cycling attack. The attacker assumes some permutations for the ciphertext. If this assumption is true, they can try and reverse the process to generate the plaintext using the ciphertext.

c. Unconcealed Message Attack

There are some rare times when, for some reason, the encrypted ciphertext is the same as the plaintext. The plaintext isn't concealed and this type of attack is called an unconcealed message attack.

2. Chosen Cipher Attack

A Chosen Ciphertext Attack (CCA) is a type of cyber-attack where a hacker chooses a specific encrypted message (ciphertext) and gets it decrypted to learn how the system works. This helps them find weaknesses and possibly break the encryption. It's a serious threat in cryptography, especially for RSA and other public key systems. Protecting against CCA is important for keeping sensitive data safe in secure communication and online transactions.

3. Factorization Attack

In a factorization attack, the attacker can impersonate the owners of the key. They can use the information to decrypt sensitive data bypassing the system's security. The attackers aim at an RSA cryptographic library. This library is used to generate the RSA key. This gives the attackers access to private keys of various security tokens, Motherboard Chipsets, and smartcards because they have the target's public key.

Blockchain - Elliptic Curve Cryptography (ECC)

Cryptography is the study of techniques for secure communication in the presence of adversarial behavior. Encryption uses an algorithm to encrypt data and a secret key to decrypt it. There are 2 types of encryptions:

1. **Symmetric-key Encryption (secret key encryption):** Symmetric-key algorithms are cryptographic algorithms that employ the same cryptographic keys both for plaintext encryption and ciphertext decoding. The keys could be identical, or there could be a simple transition between them.
2. **Asymmetric-key encryption (public key encryption):** Asymmetric-key algorithms encrypt and decrypt a message using a pair of related keys (one public key and one private key) and safeguard it from unauthorized access or usage.

Step-by-Step ECC Algorithm (Key Generation + Encryption/Decryption)

We'll break it into 3 parts:

1. **Domain Parameters (Public Setup)**

Everyone agrees on:

- Prime number ***p***
- Curve parameters ***a, b***
- Base point ***G*** (generator point)
- Order of the point ***n***

These are public values

2. **Key Generation (User Side)**

Step 1: Choose Private Key

- Select a random number:

$$d \in [1, n-1]$$

This is your **private key**

Step 2: Compute Public Key

- Multiply the base point:

$$Q = d \cdot G$$

This gives your public key

3. **Encryption (Sender Side)**

Suppose Alice sends a message to Bob.

Step 1: Represent Message as a Point

Convert message M → point $\mathbf{P_m}$ on curve

Step 2: Choose Random Number

Select random k

Step 3: Compute Cipher Points

$$C_1 = k \cdot G$$
$$C_2 = P_m + k \cdot Q$$

Ciphertext = ($\mathbf{C_1}$, $\mathbf{C_2}$)

4. **Decryption (Receiver Side)**

Bob uses private key d

Step 1: Compute Shared Secret

$$d \cdot C_1 = d \cdot (kG) = kQ$$

Step 2: Recover Message

$$Pm = C_2 - d \cdot C_1$$

Original message point is recovered!

Both sender and receiver compute the same shared secret.

But the attacker **cannot find *d*** easily.

Digital Signatures

A digital signature is a mathematical scheme that is used to verify the integrity and authenticity of digital messages and documents. It may be considered as a digital version of the handwritten signature or stamped seal. The digital signatures use asymmetric cryptography i.e. also known as public key cryptography.

What are Digital Signatures?

Digital signatures use asymmetric key cryptography. Asymmetric key cryptography also known as public key cryptography uses public and private keys to encrypt and decrypt data.

- The public key can be shared with anyone.
- The private key is the secret key that is kept a secret.

In short, it can be summarised as a digital signature a code that is attached to the message sent on the network. This code acts as proof that the message hasn't been tampered with along its way from sender to receiver.

A digital signature is intended to solve the problem of tampering and impersonation, and tampering thus it gives a recipient reason to believe:

- The message is sent by the claimed sender i.e. **Authentication**.
- The sender cannot deny having sent the message i.e. **non-repudiation**.
- The message was not altered in the transit i.e. **Integrity**.

PKI in blockchain

Public Key Infrastructure (PKI) is a security framework that issues and manages digital certificates to establish trust on digital networks. It ensures secure communication by validating identities, protecting data, and preventing unauthorized access.

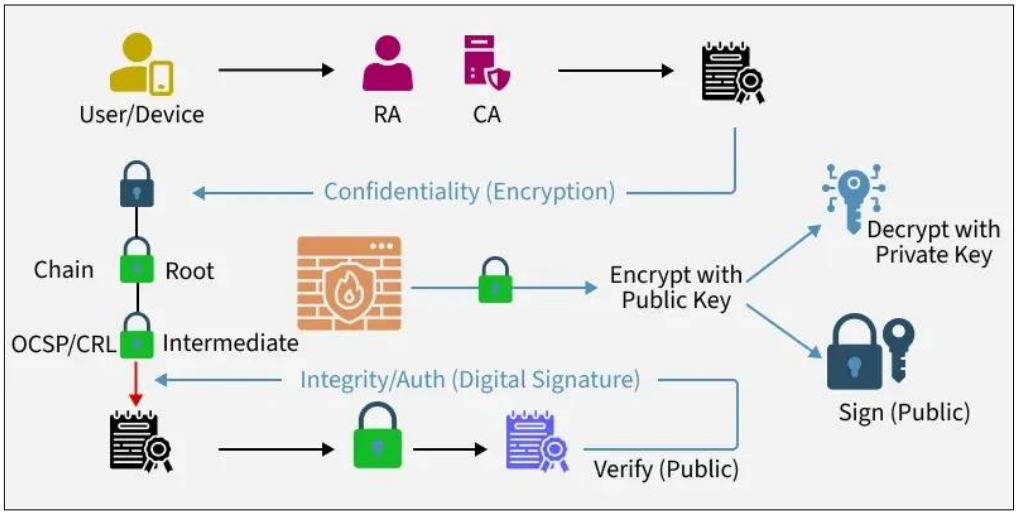
---

i. Provides unique digital identities to users, devices, and systems

ii. Secures communication using public and private key pairs

iii. Prevents MITM attacks by verifying key ownership

iv. Ensures confidentiality, integrity, and authenticity of data



Managing Keys in Cryptosystems

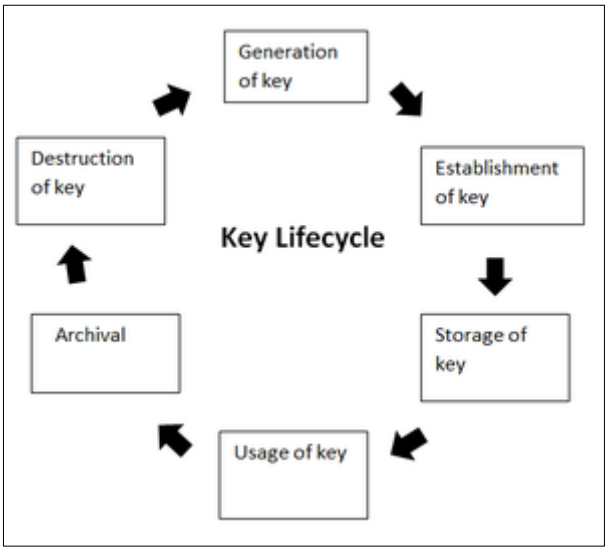
Cryptographic security depends on strong key management.

Key Areas of Key Management

Secure administration of cryptographic keys

Managing the entire key lifecycle (generation → storage → rotation → expiration → destruction)

Protecting private keys and assuring public keys



Public Key Responsibilities

Private Key Secrecy: Must stay protected and accessible only to the owner

Public Key Assurance: Public key must be verified so attackers can't replace it

PKI ensures public key validation through certificates

Components of a Public Key Infrastructure (PKI)

Digital Certificate (X.509): Contains identity + public key

Private Key Tokens: Secure storage for private keys

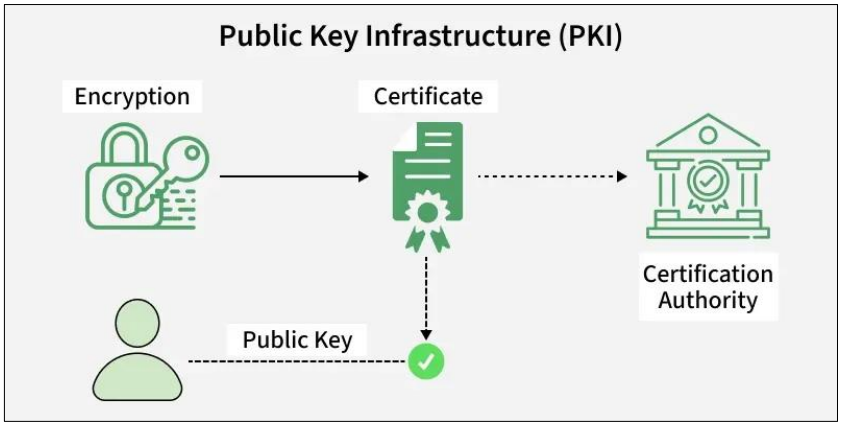
Registration Authority (RA): Verifies user identity

Certification Authority (CA): Issues and signs certificates

Certificate Management System (CMS): Handles storage and revocation

Working on a PKI

Let us understand the working of PKI in steps.



Merkle trees & Patricia tries

Merkle Trees and Patricia Tries are two core data structures used in blockchain systems (especially in Ethereum) to ensure efficiency, security, and fast verification. Let's break them down clearly and connect them to how blockchains actually work.

1. **Merkle Trees (Hash Trees)**

A hash tree is also known as Merkle Tree. It is a tree in which each leaf node is labeled with the hash value of a data block and each non-leaf node is labeled with the hash value of its child nodes labels.

A **Merkle Tree** is a tree structure where:

- Each **leaf node** = hash of a transaction
- Each **parent node** = hash of its children
- The **root** = a single hash representing all transactions

Structure

Root Hash

/ \

Hash12 Hash34

**/ \ / **

Tx1 Tx2 Tx3 Tx4

How it works

1. Hash each transaction → H(Tx1), H(Tx2), ...
2. Pair them and hash again → H(H(Tx1)+H(Tx2))
3. Repeat until one root hash (Merkle Root)

Why Merkle Trees are important?

Data Integrity

- If even 1 transaction changes → root hash changes

Efficient Verification (Merkle Proof)

- You don't need the whole dataset to verify a transaction
- Only a small path (log n complexity)

Used in:

- Bitcoin blocks (store transactions)
- Ethereum (also uses advanced versions)

2. Patricia Trie (Modified Merkle Patricia Trie)

A **Patricia Trie** (Practical Algorithm to Retrieve Information Coded in Alphanumeric) is a **compressed prefix tree** used to store key-value pairs efficiently.

In blockchain, especially Ethereum, it is upgraded into a: **Merkle Patricia Trie (MPT).**

Structure idea

Instead of storing full keys repeatedly:

- It stores **shared prefixes once**
- Compresses paths

Example:

Keys:
dog
dot

Trie:
d
|
o

/ \
g t

Wallets and Key Management

A **blockchain wallet** is not like a physical wallet. It doesn't store your cryptocurrency physically—it stores your **cryptographic keys**, which give access to your funds on the blockchain.

Think of it like:

- Wallet = **Access tool**
- Blockchain = **Bank ledger**
- Keys = **Password + Signature**

Types of Wallets

1. Hot Wallets (Online)

- Connected to the internet
- Easy to use but less secure

Examples:

- MetaMask
- Trust Wallet

2. Cold Wallets (Offline)

- Not connected to the internet
- Much more secure

Examples:

- Ledger Nano S
- Trezor Model T